English to Hindi Transformer Translation Project Documentation

This project focuses on building an English-to-Hindi neural machine translation system using the Transformer architecture in PyTorch. The notebook was designed as a learning-oriented implementation where most of the Transformer components were built manually instead of relying on high-level abstractions such as nn.Transformer or HuggingFace Trainer. The goal was not only to obtain translations, but also to understand how modern sequence-to-sequence models work internally.

1. Model Implementation

The complete translation pipeline was implemented in PyTorch. The notebook contains manual implementations of attention, masking, positional encoding, encoder blocks, decoder blocks, training loops, evaluation logic, and decoding strategies.

The model follows the classical encoder-decoder Transformer architecture introduced in the paper 'Attention Is All You Need'. The encoder processes English sentences and converts them into contextual representations, while the decoder generates Hindi tokens one word at a time.

Unlike older RNN or LSTM models that process words sequentially, the Transformer uses self-attention mechanisms that allow every word to interact with every other word in the sentence. This makes the model better at understanding long-range dependencies and contextual meaning.

2. Architecture of the Model

The architecture contains the following major components:

- Input embedding layers for both English and Hindi vocabularies
- Sinusoidal positional encoding
- Multi-head self-attention
- Feed-forward neural networks using GELU activation
- Residual connections and layer normalization
- Encoder-decoder cross attention
- Output projection layer with tied target embeddings

The notebook uses a larger educational Transformer configuration compared to tiny demo models. The implementation uses d_model = 256 along with 3 encoder layers and 3 decoder layers. The architecture was tuned for stability and quality on Google Colab T4 GPUs.

3. Dataset and Preprocessing

The dataset used in the notebook is an OPUS-style English-Hindi parallel corpus containing aligned sentence pairs. Separate train, development, and test files were used.

A major part of the project involved cleaning and preprocessing the data carefully. The notebook performs strict UTF-8 validation, removes duplicate sentence pairs, filters noisy samples, removes URL-heavy or email-heavy rows, handles hidden Unicode characters, normalizes whitespace, and removes badly aligned translations.

The preprocessing pipeline was intentionally conservative because noisy translation datasets can seriously affect training quality, especially for smaller models trained from scratch.

4. Tokenization and Vocabulary

Instead of using modern subword tokenizers such as SentencePiece or BPE, the project uses custom regex-based tokenization. English words, punctuation, numbers, contractions, and Hindi Devanagari characters are separated manually.

Separate vocabularies were created for English and Hindi. Each token was mapped to an integer ID. Special tokens such as <pad>, <unk>, <bos>, and <eos> were added for padding, unknown words, sentence beginnings, and sentence endings.

5. Training Methodology

The model was trained using teacher forcing. During training, the decoder receives the correct previous Hindi token instead of its own prediction. This stabilizes learning and improves convergence.

Cross-entropy loss with label smoothing was used as the optimization objective. The notebook also includes several practical training improvements such as gradient accumulation, gradient clipping, mixed precision training (AMP), learning-rate warmup scheduling, and GPU memory monitoring.

Training was designed for Colab T4 GPUs, where memory efficiency and training stability are important.

6. Hyperparameters

Important hyperparameters used in the notebook include:

- d_model = 256
- 3 encoder layers
- 3 decoder layers
- GELU feed-forward activation
- Label smoothing = 0.1
- Learning rate = 7e-4
- Warmup steps = 8000
- Maximum sequence length = 64
- Gradient clipping = 1.0
- Mixed precision training enabled

Beam search decoding was also implemented to improve translation quality during inference.

7. Evaluation Pipeline

The notebook evaluates translation quality using multiple metrics rather than relying on only one score. BLEU score and chrF score were used to measure translation similarity, while perplexity and validation loss were used to monitor training quality.

Both greedy decoding and beam-search decoding were implemented. Beam search produced noticeably better translations because it keeps multiple candidate sequences during generation instead of choosing only one path.

8. Results and Observations

The final model achieved meaningful translation performance considering that it was trained completely from scratch on a relatively limited dataset.

Final observed metrics from the notebook were approximately:

- Final training loss: 2.5261
- Final development loss: 4.3955
- Development BLEU score: ~12.9
- Test BLEU score: ~13.8
- chrF score: around 22–25 depending on split

The model clearly learned important translation patterns and contextual relationships. Generated outputs showed that the Transformer was capable of understanding sentence structure and semantic alignment to a reasonable extent.

However, the outputs were still far from production-quality translation systems. Some translations contained grammatical mistakes, repeated words, incorrect tense handling, or weak fluency. This is expected because the model was trained from scratch without pretrained language knowledge.

9. Challenges Faced

One of the biggest challenges was training stability. Transformers are data-hungry models and training them on smaller datasets can easily lead to overfitting or unstable convergence.

Another challenge was handling Hindi morphology and sentence structure. Hindi translations often depend on gender, tense, context, and flexible word ordering, which makes translation more difficult than simple word substitution.

GPU memory limitations on Colab T4 also required careful tuning of batch sizes, gradient accumulation, and mixed precision training.

10. Comparison with RNN/LSTM Models

Although the project focused mainly on Transformers, the architecture can still be compared conceptually with older RNN and LSTM-based translation systems.

RNNs and LSTMs process words sequentially, which makes training slower and creates difficulty in handling very long dependencies. Transformers solve this using self-attention, where every token can directly attend to every other token.

For translation tasks, this gives Transformers a major advantage in contextual understanding and parallelization. However, Transformers usually require larger datasets and more computational power compared to smaller RNN models.

11. Possible Future Improvements

The project can be improved significantly by using subword tokenization techniques such as BPE or SentencePiece. Using pretrained multilingual models like mBART, mT5, or IndicTrans would likely increase translation quality dramatically.

Larger datasets, deeper Transformer stacks, better regularization, checkpoint averaging, and longer training could also improve BLEU scores and translation fluency.

12. GitHub Repository Requirements

The final GitHub repository for the project should contain:

- Complete source code
- Jupyter notebooks and scripts
- Saved trained model checkpoints
- Dataset preprocessing pipeline
- Documentation and project explanation
- Evaluation outputs and sample translations

A proper README file explaining how to train and evaluate the model should also be included.

13. Overall Conclusion

Overall, this project demonstrates a strong educational implementation of a Transformer-based neural machine translation system in PyTorch. The notebook does not hide the internal mechanics of attention or sequence modeling, making it valuable from a learning perspective.

The project successfully combines preprocessing, Transformer implementation, training, decoding, and evaluation into a complete end-to-end translation pipeline. Even though the

final BLEU score is modest compared to industrial systems, the model clearly learns meaningful language relationships and demonstrates the practical working of modern NLP architectures.